Universidad Tecnológica Nacional — Facultad Regional Mar del Plata
Tecnicatura Universitaria en Programación
Programación IV · Trabajo Práctico Final



Bitácora técnica

Stack, reglas de arquitectura, modelo de dominio y errores resueltos

Autor	Eloy Roldán
Proyecto	AlojaME — plataforma de alquileres temporarios
Repositorio	github.com/EloyRolda/tp_gestion_alojamientos
Versión y fecha	1.0 · Noviembre de 2026

Índice

1 Introducción 2

2 Stack tecnológico 3

2.1 Backend 3

2.2 Frontend 3

3 Reglas de arquitectura 5

3.1 Backend 5

3.2 Frontend 6

4 Estructura del código 7

4.1 Backend 7

4.2 Frontend 7

5 Modelo de dominio 9

5.1 Reserva 9

5.2 Pago 9

6 Seguridad y roles 11

7 Registro y verificación de cuentas 12

8 Errores resueltos del backend 13

9 Errores resueltos del frontend 17

1 Introducción

Esta bitácora reúne la información técnica de referencia de AlojaME: las tecnologías utilizadas, las reglas de arquitectura que sigue el código, el modelo de dominio y el historial de errores encontrados durante el desarrollo, con su causa y su solución, para no volver a cometerlos. Las decisiones de diseño del frontend se explican en la *Documentación del frontend*.

2 Stack tecnológico

2.1 Backend

Componente	Tecnología
Lenguaje y framework	Java 21, Spring Boot 3.4.3
Persistencia	MariaDB / MySQL, JPA / Hibernate
Seguridad	Spring Security, JWT (JJWT 0.12.6), sin sesión en el servidor
Servicios externos	MercadoPago (cobros, reembolsos y liquidaciones), Cloudinary (imágenes), Brevo (correo de verificación)
Documentación de la API	Springdoc OpenAPI (Swagger)
Utilidades	Lombok, Bean Validation

- Paquete base: `com.eloyroldan.tp_final_alojamiento.backend`. Base de datos por convención: `testdb`.
- `spring.jpa.hibernate.ddl-auto=update`: las tablas y columnas nuevas las genera Hibernate. Excepción: las columnas `GENERATED ALWAYS AS` (por ejemplo, `reserva.precio_total`) y las restricciones `CHECK` no las puede generar desde cero (ver la sección de errores resueltos).
- `spring.jpa.open-in-view=false`: los controladores nunca devuelven entidades JPA; todo sale como DTO armado dentro de métodos `@Transactional`.

2.2 Frontend

Componente	Tecnología
Framework	Angular 22.2 (componentes *standalone*, signals, router con carga diferida)
Lenguaje	TypeScript 6
Programación reactiva	RxJS 7.8 (solo en los servicios HTTP)
Gráficos	Chart.js 4
Mapas	Leaflet 1.9 con OpenStreetMap
Estilos	CSS propio, sin librerías de componentes
Compilación	Angular CLI con `@angular/build` (esbuild)
Requisitos	Node.js 22.22 o superior

- La URL de la API se lee de `public/env.js` en tiempo de ejecución, sin recompilar.
- La carga inicial de la aplicación se mantiene por debajo de 500 kB: las pantallas, Chart.js y Leaflet se descargan cuando se usan.

3 Reglas de arquitectura

3.1 Backend

Un *Service* solo puede usar su propio *Repository*, nunca el de otra entidad. Si necesita datos de otra tabla, llama al *Service* dueño de esa tabla.

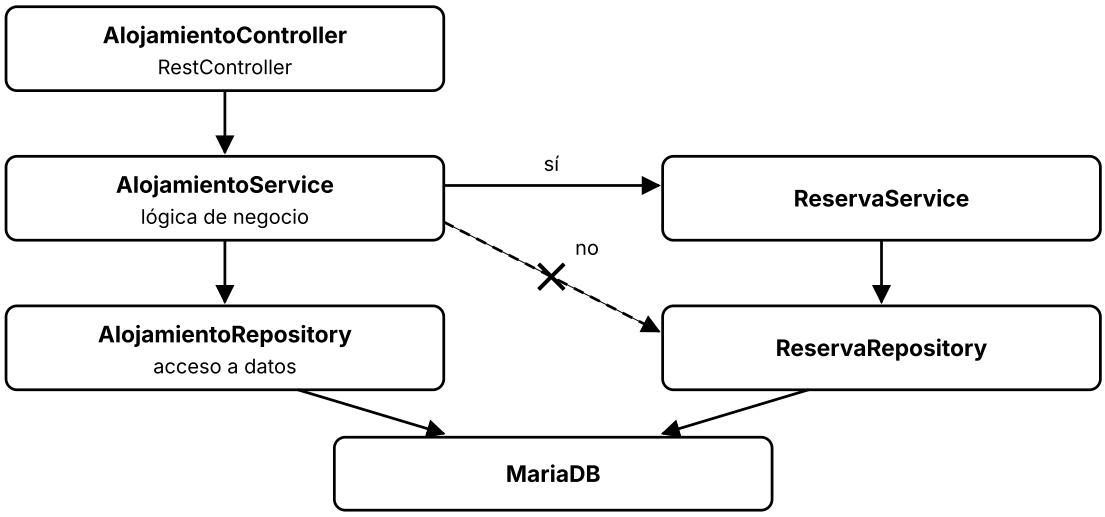


Figura 1. Regla de capas del backend: para usar datos de otra entidad se llama a su Service, nunca a su Repository.

Los ciclos de dependencia entre *Services*, inevitables en algunos casos, se rompen con `@Lazy` del lado que menos lo necesita:

- `AlojamientoService` ↔ `ReservaService` (`@Lazy` en `AlojamientoService`).
- `PagoService` ↔ `ReservaService` (`@Lazy` en `PagoService`, porque `ReservaService` depende de `PagoService` en casi todas sus transiciones de estado).
- `UsuarioService` ↔ `VerificacionService` (`@Lazy` en `UsuarioService`).
- El mismo criterio para `TipoAlojamientoService` ↔ `AlojamientoService`, `DireccionService` ↔ `AlojamientoService` y `ServicioService` ↔ `ServicioxAlojamientoService`.

Para verificarlo se buscan las declaraciones `private final ...Repository` de cada *Service* y se comparan con el nombre del propio *Service*.

Políticas de cancelación (patrón Strategy). `PagoService.ejecutarCancelacion` no contiene la lógica de reembolso de cada motivo. Cada motivo tiene su propia clase en `Service/Cancelacion/` que implementa `PoliticaCancelacion` (`ReembolsoTotalPolicy`, `RetencionTardiaPolicy`, `ReembolsoManualAdminPolicy`). `PagoService` las recibe como `List<PoliticaCancelacion>` y las indexa por motivo: agregar un motivo nuevo es agregar una clase, nunca un `switch`.

Pasarela de pago (inversión de dependencias). `PagoService` y las políticas dependen de la interfaz `PasarelaDePagoService` (`crearPreferencia`, `consultarPago`, `reembolsar`, `crearPayout`), no de `MercadoPagoService`. Cambiar de proveedor de pagos es agregar una implementación de esa interfaz. `PasarelaSimuladaService` es una segunda implementación, activa solo con `PAGOS_SIMULADOS=true`, que registra reembolsos y liquidaciones sin llamar a MercadoPago.

Redondeo monetario. Todo redondeo de dinero pasa por `Service/MontoUtil.redondear()`.

3.2 Frontend

Las pantallas nunca llaman a la API directamente: lo hacen a través de un servicio por recurso, y el interceptor agrega el token a cada petición.

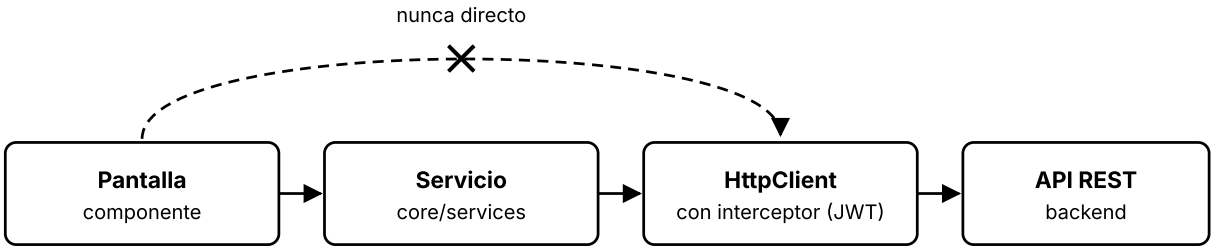


Figura 2. Capas del frontend: las pantallas piden los datos a un servicio; solo los servicios usan HttpClient.

- **Componentes standalone con signals.** El estado de cada pantalla vive en `signal()` y lo derivado en `computed()`; no se usan `NgModules`.
- **Un servicio por recurso de la API** en `core/services/`. Ningún componente inyecta `HttpClient`.
- **Un único lugar para cada responsabilidad transversal:** el token y la sesión vencida en el interceptor; los mensajes de error en `http-error.util.ts`; las fechas en `fecha.util.ts`; las reglas de cancelación que se muestran en pantalla en `cancelacion.util.ts`; las confirmaciones y avisos en `ConfirmacionService`.
- **Rutas protegidas.** Toda ruta que requiere sesión o rol declara su guard en `app.routes.ts`, y las pantallas de edición verifican además que el recurso sea del usuario.
- **Estilos.** Los colores y las medidas existen solo como variables de `src/styles.css`; las utilidades responsive (`.form-grid`, `.table-cards`, `.kpi-grid`) se reutilizan en lugar de escribir CSS suelto.
- **Carga diferida.** Las pantallas y las librerías pesadas se descargan cuando se usan.
- **Documentación en el código.** Cada archivo comienza con un comentario `///` que explica su propósito.

4 Estructura del código

4.1 Backend

Paquete	Contenido
`Config/`	`SecurityConfig`, `JwtAuthFilter`, `CustomUserDetailsService`, `CloudinaryConfig`.
`Converter/`	Conversores de enumerados (la base guarda minúsculas; Java usa mayúsculas).
`DTO/`	DTOs de entrada y, en `Response/`, los de salida.
`Enum/`	`ESTADOPAGO`, `ESTADORESERVA`, `TIPOUSUARIO`, `MOTIVOCANCELACION`.
`Exception/`	`GlobalExceptionHandler` y excepciones propias.
`Model/`	Entidades JPA (Alojamiento, Reserva, Pago, Usuario, Review, etc.).
`Repository/`	Un repositorio por entidad.
`RestController/`	Endpoints REST.
`Service/`	Lógica de negocio: un *Service* por entidad, más orquestadores como `EstadisticasController`, que combina varios *Services* sin ser dueño de ninguna tabla.
`Validation/`	Validaciones propias (por ejemplo, `DominioEmailValido`).
`db/` (fuera de `src`)	Scripts SQL: catálogo inicial, datos de prueba y de demostración.

4.2 Frontend

Carpeta	Contenido
`core/config/`	Configuración de la URL de la API y del modo de pagos simulados.
`core/guards/`	`authGuard`, `roleGuard`, `soloVisitanteGuard`, `idNumericoGuard`.
`core/interceptors/`	`authInterceptor`.
`core/models/`	Interfaces TypeScript que reflejan los DTOs del backend.
`core/services/`	Servicios HTTP, `AuthService`, `ConfirmacionService` y `BajaUsuarioService`.
`core/utils/`	Fechas, mensajes de error, cancelaciones, búsqueda, imágenes y control de dueño.
`shared/`	Componentes reutilizables (carta de avisos, calendario, gráficos, mapa, Kaza, etc.).
`layout/`	Encabezado, pie y páginas 403 y 404.

Carpeta	Contenido
`features/`	Pantallas agrupadas por perfil: `public`, `auth`, `cliente`, `anfitrion`, `admin`.

5 Modelo de dominio

5.1 Reserva

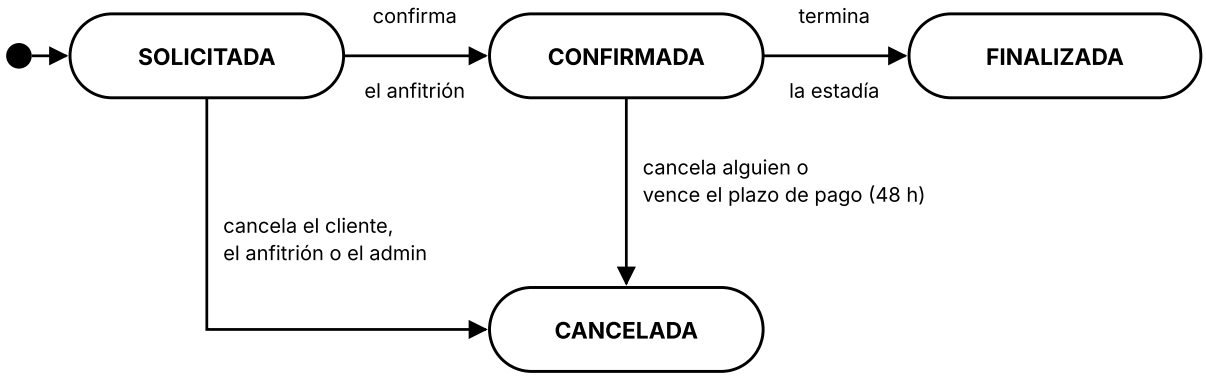


Figura 3. Máquina de estados de una reserva.

- El cliente crea la reserva en estado `SOLICITADA`.
- El anfitrión (o el administrador) la confirma: pasa a `CONFIRMADA`, comienza el plazo de pago de 48 horas (`fecha_confirmacion`) y se crean el `Pago` y la preferencia de MercadoPago.
- Si no se paga en 48 horas, una tarea programada (`PagoService.cancelarPagosVencidos`) la cancela.
- Pasa a `FINALIZADA` cuando terminó la estadía y el anfitrión (o el administrador) la marca; eso dispara la liquidación al anfitrión.
- Para evitar que dos solicitudes simultáneas sobre el mismo alojamiento pasen juntas el control de superposición de fechas, crear o reprogramar una reserva usa `AlojamientoService.obtenerPorIdConLock()` (bloqueo pesimista `PESSIMISTIC_WRITE`).

5.2 Pago

La plataforma retiene el dinero y lo liquida al anfitrión.

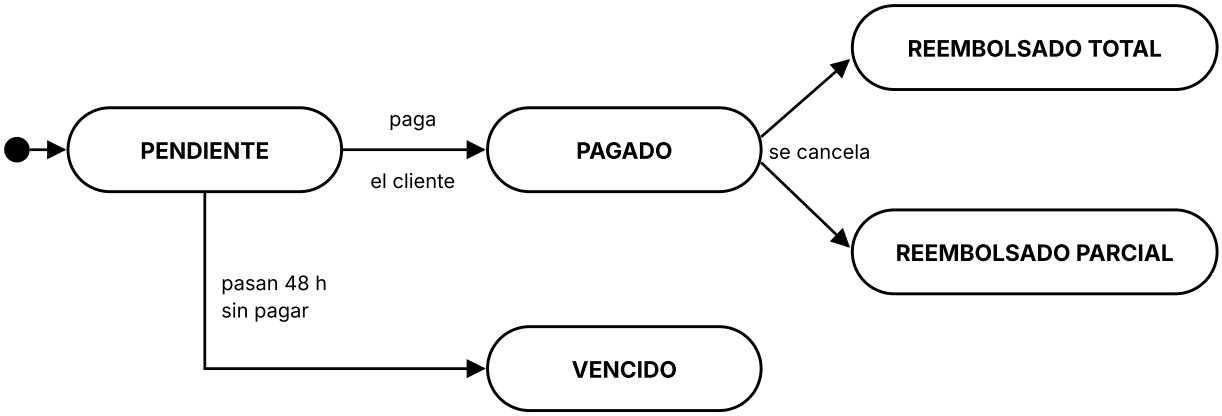


Figura 4. Máquina de estados de un pago. El reembolso parcial corresponde a la cancelación tardía o al reembolso a medida del administrador.

- El anfitrión define el `monto_base` (lo que quiere recibir). El cliente paga `monto_base` + comisión de la plataforma (5 %) + comisión estimada de MercadoPago (6 %, configurable).
- El anfitrión nunca opera MercadoPago: carga un alias, CBU o CVU (`Usuario.aliasCobro`) validado con el dígito verificador del BCRA y la identificación del banco (`AliasCobroService`).
- Política de cancelación (`ReservaService.determinarMotivo`):
  - Cancela el anfitrión: reembolso del 100 % al cliente.
  - Cancela el cliente con 48 horas o más de anticipación: reembolso del 100 %.
  - Cancela el cliente con menos de 48 horas: reembolso del 65 %; el anfitrión recibe el 35 % restante.
  - Vence el plazo de pago: no se mueve dinero y el pago queda `VENCIDO`.
  - Administrador: reembolso a medida, validado para que no supere el `monto_base`.
- Las comisiones (plataforma y MercadoPago) no se devuelven en ningún caso.
- La liquidación al anfitrión ocurre en dos momentos: al finalizar la estadía (100 %) o ante una cancelación tardía (35 %).

6 Seguridad y roles

Tres roles: `ADMIN`, `ANFITRION` y `CLIENTE`, definidos en `SecurityConfig` con JWT y sin sesión en el servidor. El control de propiedad de un recurso (por ejemplo, que un cliente solo pueda cancelar sus propias reservas) no vive en `SecurityConfig`: lo resuelve cada *Service* o controlador comparando identificadores.

7 Registro y verificación de cuentas

El usuario se registra con `email_verificado=false` y no puede iniciar sesión hasta confirmar su cuenta mediante un token de un solo uso (válido 24 horas, tabla `token_verificacion`) que se envía por correo con Brevo. Los administradores se crean ya verificados. La validación del dominio del correo es híbrida: una lista de proveedores habituales y, para el resto, una consulta MX por DNS (`DominioEmailValidoValidator`).

8 Errores resueltos del backend

1. `reserva.precio_total` es una columna `GENERATED ALWAYS AS (precio_noche_pactado * DATEDIFF(fecha_fin, fecha_inicio))`, pero Hibernate nunca la puede recrear desde cero (no tiene anotación para eso) y en MariaDB 10.4 un `ALTER TABLE` sobre otra columna puede hacer que la tabla se reconstruya y esa columna quede en null. Por eso `PagoService.calcularMontoBase()` recalcula noches×precio en Java y jamás confía en `Reserva.getPrecioTotal()`. No volver a depender de esa columna para nada crítico.
2. `JwtAuthFilter` corre antes del `DispatcherServlet`, fuera del alcance de `GlobalExceptionHandler`. Un JWT vencido o manipulado tira `JwtException`/`IllegalArgumentException` sin capturar si no se blinda ahí mismo con try/catch — ya está resuelto, pero cualquier cambio a ese filtro tiene que mantener ese try/catch.
3. Los enums se guardan en minúscula en la BD, en mayúscula en Java. Se resuelve con `AttributeConverter` (no `@Enumerated`), no volver a usar `@Enumerated` directo.
4. FK sin cascada entre `pago` e `reserva` (`pago.id_reserva → reserva.id_reserva`, sin `ON DELETE CASCADE`). Cualquier borrado de una `Reserva` (individual o en cascada desde `Alojamiento`) tiene que borrar el `Pago` asociado antes (`PagoService.borrarPorReserva`/`borrarPorAlojamiento`), o la FK lo rechaza.
5. N+1 queries: los listados de `Alojamiento`, `Reserva` y los resúmenes de `Estadisticas` usan `JOIN FETCH` o consultas agrupadas (`Map<Long, X>` con una sola query agrupada) en vez de una query por fila. Si se agrega un nuevo listado, replicar ese patrón, no iterar y consultar de a uno.
6. `/actuator/prometheus` requiere la dependencia `micrometer-registry-prometheus` además de `spring-boot-starter-actuator` — sin eso, el endpoint no se registra aunque esté en `management.endpoints.web.exposure.include`. **Ya agregada** en el `pom.xml` (se detectó que faltaba pese a estar documentada: el endpoint devolvía 404).
7. Índices y `CHECK` constraints se declaran vía anotaciones JPA (`@Table(indexes=...)`, `@Check` de Hibernate) en los Models — así Hibernate los genera solo en cualquier base nueva. `db/completar_indices_y_checks.sql` es el fallback manual, de una sola vez, solo para bases que ya existían antes de agregar esas anotaciones.
8. `JwtAuthFilter` validaba la firma/vencimiento del token pero no `userDetails.isEnabled()`: una cuenta recién desactivada (`Usuario.activo=false`) seguía autenticando con un JWT emitido antes de la baja hasta por 24hs. **Solución:** el filtro ahora exige `isTokenValid(...) && userDetails.isEnabled()`.
9. `confirmar`/`cancelar`/`cancelarComoAdmin`/`finalizar` de `ReservaService` leían la `Reserva` con `obtenerPorId` (sin lock): dos transiciones de estado concurrentes sobre la misma reserva podían pasar juntas el chequeo de estado y disparar el Payout/reembolso dos veces. **Solución:** usan `obtenerPorIdConLock` (`ReservaRepository.findByIdConLock`, `PESSIMISTIC_WRITE`), mismo patrón que `AlojamientoService.obtenerPorIdConLock`.

10. `application.properties` tenía credenciales escritas directamente en el archivo (DB, Cloudinary, JWT secret, Brevo, MercadoPago) — algunas como default de `${VAR:...}`, otras directo. **Solución:** las 6 dependen de su variable de entorno sin ningún default — la app falla al arrancar si falta alguna, en vez de arrancar con un secreto comprometido. En producción se usan credenciales nuevas (Cloudinary, Brevo, MercadoPago y un `jwt.secret` propio), nunca las de desarrollo.
11. CORS tenía `setAllowedOriginPatterns(List.of("*"))` combinado con `allowCredentials(true)`: cualquier sitio externo podía mandar requests autenticados (JWT) contra esta API desde el browser del usuario. **Solución:** `cors.allowed-origins` (lista explícita, por defecto solo `app.frontend-url`).
12. `ReservaService.cancelar`/`cancelarComoAdmin` llamaban siempre a `PagoService.ejecutarCancelacion`, que asume que ya existe un `Pago` (solo se crea en `confirmar()`). Cancelar una reserva todavía `SOLICITADA` (transición válida según el diagrama de arriba) tiraba `IdNoEncontradoException` ("Esta reserva todavía no tiene un pago asociado") en vez de cancelarla — reproducido con `curl` contra una base MariaDB antes de corregirlo. La transacción rollbackeaba bien (no corrompía datos), pero la operación no funcionaba. **Solución:** ambos métodos solo llaman a `PagoService.ejecutarCancelacion` cuando la reserva ya pasó por `CONFIRMADA`.
13. **SecurityConfig** abre a lectura pública (`permitAll`, acotado con `HttpMethod.GET`) los endpoints de solo lectura para navegar el catálogo sin cuenta: listar/mostrar `Alojamiento`, sus servicios, `Galeria`, `Review` por alojamiento, y los catálogos `TipoAlojamiento`/`Servicio`. Decisión explícita del equipo (modelo tipo Airbnb: se navega libre, el login solo hace falta para publicar/reservar/cualquier escritura) — antes de esto, **ningún** endpoint de `Alojamiento` era público, ni siquiera listar. Donde un mismo path sirve lectura y escritura (ej. `/Alojamiento/{id}/servicios`: GET lista, POST agrega) el `permitAll` está acotado por método HTTP, nunca al path entero — no volver a hacerlo público sin ese acotamiento.
14. **`MercadoPagoService.crearPayout` (API de Payouts) devuelve `400 Bad Request: "Invalid signature"`** al probarlo con las credenciales de prueba de la cuenta de desarrollador del proyecto — confirmado llamando a `ReservaService.finalizar()` de punta a punta (reserva confirmada, pagada y con el webhook procesado, contra la API de MercadoPago). `crearPreferencia`, `consultarPago` y el webhook **sí** funcionan perfecto con las mismas credenciales; es específicamente el endpoint `/v1/payouts` el que rechaza la llamada. Ya estaba anticipado en el comentario original de `crearPayout` ("hay que confirmarlo contra el panel de developers... misma limitación de sandbox") — la API de Payouts necesita habilitación extra (cuenta marketplace/collector) que esta app de prueba no tiene. **No es un bug de este código.** Si se vuelve a probar `finalizar()`/el Payout de cancelación tardía con credenciales nuevas, probablemente haga falta habilitar Payouts específicamente en el panel de MercadoPago antes de que funcione.
15. **`TipoAlojamientoController`/`ServicioController` solo tenían `listar`/`reemplazar`/`eliminar`, sin alta ni edición en el lugar** — detectado al construir las pantallas del administrador (SCRUM-43/44 piden explícitamente "alta, edición y baja" del catálogo). Se agregaron `POST /crear` y `PUT /{id}` (`NombreCatalogoDTO { nombre }`) a ambos controllers, con los métodos `crear`/`renombrar` correspondientes en `TipoAlojamientoService`/`ServicioService` (mismo criterio de normalización

que `obtenerOcrear`, pero sin reutilizar en silencio: `crear` tira error si ya existe un nombre igual, `renombrar` tira error si el nuevo nombre colisiona con otra fila — ahí corresponde `reemplazar`, no `renombrar`). `SecurityConfig` actualizado: `POST /TipoAlojamiento/crear` y `POST /Servicio/crear` van con el resto de los matchers `hasRole("ADMIN")` de cada catálogo; `PUT /TipoAlojamiento/*` y `PUT /Servicio/*` son matchers nuevos, acotados por `HttpMethod.PUT`, también `ADMIN`. Verificado: alta, edición, y rechazo con mensaje claro al intentar crear/renombrar con un nombre que ya existe.

16. **`POST /auth/reenviar-verificacion` era un oráculo de enumeración de usuarios** — detectado en la auditoría de lógica de negocio. Antes de la correción, `VerificacionService.reenviarVerificacion` tiraba una excepción distinta segun el caso: `UnautorizedExeption("Usuario no encontrado: <email>")` si el email no existía, `ParametroInvalidoException("Esta cuenta ya fue verificada.")` si existía y ya estaba verificado, o éxito si existía y no estaba verificado — las 3 respuestas son distintas y públicas (sin login), asi que cualquiera podía confirmar, probando emails al voleo, cuáles están registrados en la plataforma y cuáles ya verificaron su cuenta. El login (`/auth/login`) no tiene este problema: Spring Security ya normaliza `UsernameNotFoundException` a `BadCredentialsException` (`hideUserNotFoundExceptions`, default `true`), asi que usuario inexistente y contraseña incorrecta devuelven el mismo mensaje — confirmado con curl. **Solución:** se agregó `UsuarioService.buscarPorEmail` (devuelve `Optional`, no tira excepción) y `reenviarVerificacion` ahora siempre responde igual sin importar el caso (si el email no existe, o ya está verificado, simplemente no hace nada; solo genera y envía un token nuevo cuando corresponde). El Controller nunca ve la diferencia.
17. **Cambiar el propio email en "Mi perfil" deslogueaba a mitad de sesión, sin ningún error visible** — detectado en la auditoría de lógica de negocio, reproducido con pruebas automatizadas de navegador (Playwright). Causa: `CustomUserDetailsService` usa siempre el **email** como `username` interno del `UserDetails`, y `JwtService.generateToken` firma el JWT con ese username como `subject` — asi que el JWT emitido al loguearse tiene el email viejo. `JwtAuthFilter` resuelve el usuario en cada request autenticado llamando a `userDetailsService.loadUserByUsername(subjectDelToken)`; en cuanto `UsuarioService.actualizar` pisa el email en la fila de `usuario`, esa búsqueda por el email viejo deja de encontrar a nadie y el filtro simplemente no autentica el request (no tira ningún error visible: la próxima request autenticada se trata como anónima y termina en 401/redirect a `/login`). **Solución:** `UsuarioController.actualizar` detecta si el email cambió y, si cambió, reemite un JWT nuevo (con el email nuevo como subject) y lo devuelve en el header de respuesta `X-New-Token`; `CorsConfiguration` necesitó `setExposedHeaders(List.of("X-New-Token"))` porque sin eso el browser descarta el header antes de que el JS del front lo vea (`setAllowedHeaders` es para headers de request, no de response). Del lado del frontend, `UsuarioService.actualizar` pide la respuesta con `observe:'response'`, lee ese header, y si está presente llama a `AuthService.actualizarToken(...)` (método nuevo, wrapper público de `guardarSesion`) para reemplazar el JWT guardado de forma transparente — el usuario nunca se entera de que hubo un token nuevo. Verificado: cambiar el email

ya no corta la sesión, la navegación siguiente (`/mis-reservas`) sigue autenticada. Si se agrega otro campo que participe de la identidad del `UserDetails` (hoy solo el email) y se vuelve editable, aplicar el mismo patrón.

18. **No hay rate limiting / bloqueo por intentos fallidos en `/auth/login`** — detectado en la auditoría de lógica de negocio. Nada frena hoy un ataque de fuerza bruta probando contraseñas contra un `identificador` conocido (no hay `Bucket4j` ni librería de rate limiting en el `pom.xml`, ni ningún contador de intentos fallidos). **Decisión explícita del equipo: no implementarlo** — queda documentado como gap conocido, no bloqueante para el alcance de este TP universitario. Si se retoma: un filtro liviano en memoria (ej. 5 intentos fallidos por `identificador`+IP en una ventana de 15 min, bloqueo temporal) alcanza sin agregar dependencias nuevas.

9 Errores resueltos del frontend

1. **`NG0200: Circular dependency detected for _AuthService`** al recargar la página con sesión activa (`AuthService` es `providedIn:'root'`). Causa: el constructor de `AuthService` llamaba `cargarPerfil()` (un `GET /Usuario/me`) de forma sincrónica cuando ya había un token guardado; esa request pasa por `authInterceptor`, que hace `inject(AuthService)` — y como el constructor todavía no había terminado de devolver, Angular lo detecta como dependencia circular contra sí mismo. El error ni siquiera llega a salir a la red (se puede confirmar con `page.route()` de Playwright: la request nunca se dispara), cae directo en el `catchError` de `cargarPerfil()`, que llama a `cerrarSesion()` — **el usuario pierde la sesión en cada F5**, silenciosamente, sin ningún error visible en la UI. Reproducido y diagnosticado con Playwright (instrumentando `Storage.prototype.setItem/removeItem` para loguear el stack trace exacto que disparaba el `removeItem`). **Solución:** la llamada inicial a `cargarPerfil()` se difiere con `queueMicrotask(...)`, así corre después de que el constructor ya devolvió y el injector tiene la instancia completamente registrada. Si se agrega otro servicio `providedIn:'root'` cuyo constructor dispare una request HTTP de entrada (no a pedido de un componente), aplicar el mismo patrón.
2. **Prueba del flujo de pago contra el entorno de prueba de MercadoPago.** El pago se completa a mano en un navegador, iniciando sesión en MercadoPago con una cuenta de **comprador de prueba** (nunca con la cuenta vendedora de las credenciales, porque MercadoPago lo rechaza con "una de las partes es de prueba") y con las tarjetas de prueba oficiales. En desarrollo, la `notification_url` apunta a `localhost:8081`, que no es accesible desde internet, de modo que la notificación de MercadoPago no llega: se simula llamando a `POST /Pago/webhook` con `type=payment` y `data.id=<id de la operación>`. Esa llamada ejecuta el mismo código de producción (`procesarNotificacionPago`), que vuelve a consultar el estado del pago a la API de MercadoPago en lugar de confiar en el cuerpo recibido. En el servidor, con HTTPS público, la notificación llega sola.
3. **El flujo de alias de cobro (`POST /Usuario/alias-cobro/validar` y `/confirmar`) no tenía ninguna pantalla en el frontend** — detectado en la auditoría del frontend: el backend soporta el paso a paso completo (validar formato/banco sin guardar, luego confirmar y persistir) desde las etapas anteriores, pero ningún componente lo llamaba. Sin esto, un anfitrión no tiene forma de indicarle a la plataforma a dónde pagarle (bloquea el Payout aunque funcionara). **Solución:** se agregó la sección "Alias de cobro" en `mi-perfil` (visible solo para `ANFITRION`/`ADMIN`, mismo patrón de `rol()` que el navbar), con el flujo de 2 pasos (`UsuarioService.validarAliasCobro`/`confirmarAliasCobro`) y el input sin atributo `name`. Verificado con un CBU válido: identificó el banco (BBVA Argentina), mostró la advertencia de no poder confirmar el titular, y persistió el alias tras confirmar.